toXo

v2.0

Task Management and Personal Finance
Desktop Application

Technical Documentation

Complete guide to architecture, features, and usage

Author: TheOverforge | github.com/TheOverforge

Technology Stack

Electron 31 **React 18** **TypeScript 5**

Vite 5 Zustand 4 better-sqlite3 9

TipTap · Recharts · Radix UI · Tailwind CSS · i18next

April 19, 2026 | Windows 10/11

Contents

1 Application Overview **3**
1.1 Key Features 3

2 Technology Stack **4**
2.1 Electron 31 4
2.2 React 18 + TypeScript 5 5
2.3 Vite 5 5
2.4 Zustand 4 5
2.5 better-sqlite3 9 6
2.6 TipTap 2 6
2.7 Recharts 2 6
2.8 Additional Dependencies 6

3 Architecture **7**
3.1 Monorepo Structure 7
3.2 Data Flow 8
3.3 Main Process 8
3.3.1 index.ts — App Lifecycle 8
3.3.2 windowManager.ts — Window and Visual Effect . . 9
3.3.3 notificationScheduler.ts — Notifications 9
3.4 Preload 9
3.5 Renderer 9

4 Database **10**
4.1 File Paths 10
4.2 Table Schema 10
4.2.1 tasks 10
4.2.2 subtasks 10
4.2.3 categories 11
4.2.4 Finance Tables 11
4.3 AppDatabase Class 11

5 Application Pages **12**
5.1 Tasks 12
5.1.1 Multi-select 12
5.1.2 Inline Rename (F2) 12
5.1.3 Task Context Menu 12
5.1.4 Undo/Redo 13
5.2 Analytics 13
5.3 Calendar 13

5.4 Finance 14
5.5 Settings 14

6 IPC API Reference 14
6.1 tasks 14
6.2 subtasks 15
6.3 Other Channel Groups 16
6.4 Push Events (main → renderer) 16

7 Keyboard Shortcuts 16

8 Build and Distribution 17
8.1 Scripts 17
8.2 electron-builder 17
8.3 Fresh Machine Setup 17

9 Settings File (settings.json) 18

1 Application Overview

toXo is a Windows desktop application for personal task management and finance tracking. Starting with version 2.0 the application has been completely rewritten: Python + PyQt6 has been replaced by the **Electron 31 + React 18 + TypeScript** stack, while the SQLite database format is fully preserved — existing data from previous versions loads without any migration. The interface supports three themes: **Dark**, **Light**, and **Glass** (native acrylic-blur DWM). The accent color is `#0a84ff`; documentation is styled in the author's signature purple palette.

1.1 Key Features

- **Full task CRUD** — create, view, edit, delete
- **Rich-text editor** — TipTap: bold / italic / strikethrough, H1–H3 headings, lists, quotes, code blocks, custom colors and fonts
- **Priorities** — three levels (1 low, 2 medium, 3 high) with color indicators
- **Pinning** — pin tasks to the top of the list
- **Deadlines** — notification the day before and on the deadline day
- **Reminders** — arbitrary time, native OS notification
- **Recurrence** — daily / weekly / monthly
- **Tags** — arbitrary comma-separated labels
- **Subtasks** — checkbox list with drag-and-drop reordering
- **Categories** — colored folders; drag-and-drop for tasks and folders
- **Filtering** — by status (all / active / done / today / archive), category, and search query
- **Sorting** — by date (newest / oldest), alphabet, undone-first, manual
- **Archive** — manual and auto-archiving of completed tasks
- **Undo/Redo** — full history via Ctrl+Z / Ctrl+Y
- **Duplicate** — Ctrl+D copies the task with all its fields
- **Multi-select** — Ctrl+click adds/removes a task, Shift+click selects a range, Ctrl+A selects all; bulk actions via BulkActionBar
- **Inline rename** — F2 or double-click on a task name puts the card into inline edit mode
- **Analytics page** — 6 Recharts graphs, KPI cards, streak, periods 7/30/90 days and **custom date range**

- **Calendar page** — monthly grid with deadline chips, day panel, quick create/edit, **drag tasks onto cells** (drag-to-navigate via sidebar), open-in-tasks button (↗)
- **Finance page** — accounts, transactions (income/expense/**transfer between accounts**), categories, budgets, savings goals, 5 charts
- **Command palette** — Ctrl+K, search and run any command
- **Splash screen** — animated loading screen on startup
- **First-run** — automatic welcome task in the app's current language
- **Interactive tutorial** — 11 steps with automatic page navigation, element spotlight, animated transitions
- **Localization** — Russian and English, live language switching
- **System tray** — background operation, quick actions from tray menu
- **Three themes** — Dark, Light, Glass (acrylic blur)
- **Notification sounds** — configurable reminder sound
- **Export/Import** — JSON and CSV
- **Demo data** — pre-filled tasks and transactions (language-aware)

2 Technology Stack

2.1 Electron 31

The application shell. Electron provides a native process (main) and a browser context (renderer). Key Electron APIs used in toXo:

API	Usage
`BrowserWindow`	Main window with `transparent: true` and `frame: false`
`ipcMain.handle`	Registers request handlers from the renderer
`contextBridge`	Securely exposes API to the renderer via preload
`Notification`	Native OS notifications (reminders, deadlines)
`Tray`	System tray icon with context menu
`app.requestSingleInstanceLock`	Single application instance guard
`SetWindowCompositionAttribute`	Native acrylic-blur DWM via Win32 FFI

Important: transparent + acrylic

The window is created with `transparent: true` and `show: false`. Acrylic is applied via `SetWindowCompositionAttribute` *before* calling `win.show()` — this ensures correct DWM rendering from the very first frame, with no grey flash.

2.2 React 18 + TypeScript 5

The renderer is written in React 18 with full TypeScript typing. UI architecture:

- **Pages** — `TasksPage`, `AnalyticsPage`, `CalendarPage`, `FinancePage`, `SettingsPage`
- **Widgets** — `AppSidebar`, `TaskEditor`, `TaskList`, `CommandPalette`, `WindowControls`, `TourOverlay`, `LaunchOverlay`
- **Features** — `CategoryBar`, `Topbar`, filter components
- **Shared UI** — `Button`, `Dialog`, `DateTimePicker`, `ColorPicker` (built on Radix UI)

2.3 Vite 5

The renderer bundler. Uses `@vitejs/plugin-react` (SWC). Configuration:

- Alias `@/` → `renderer/src/`
- Dev mode: hot module replacement at `localhost:5173`
- Production: single bundle `dist/assets/index-*.js` + CSS

2.4 Zustand 4

State manager. Three primary stores:

Store	Contents
`useTaskStore`	tasks, subtasks, selectedTaskId, **selectedTaskIds**, filterMode, sortMode, searchQuery, currentCategoryId; methods: toggleSelectTask, selectAll, addToSelection, clearMultiSelect
`useCategoryStore`	categories
`useHistoryStore`	undo/redo stack — pairs of `{undo, redo}` functions

2.5 better-sqlite3 9

Synchronous SQLite in the Electron main process. Database file: `%APPDATA%\toxo\task`
Usage notes:

- `PRAGMA journal_mode = WAL` — concurrent reads without locking
- `PRAGMA foreign_keys = ON` — cascading deletes for subtasks and transactions
- Synchronous API eliminates the need for `async/await` in the repository layer
- Native `.node` module requires `asarUnpack` and `electron-rebuild`

2.6 TipTap 2

Rich-text task description editor. Extensions used:

- **StarterKit** — base formatting, headings, lists, blockquotes
- **TextStyle + Color** — arbitrary text color (palette + favorites)
- **FontFamily** — custom fonts (system fonts + user-added)
- **Underline** — underline formatting
- Content is stored as an HTML string in the `description` column of `tasks`

2.7 Recharts 2

React chart library. Used in Analytics and Finance sections:

Component	Usage
`AreaChart`	Tasks completed per day, cumulative balance
`BarChart`	Created vs completed, activity by weekday
`PieChart / RadialBarChart`	Status and priority distribution
`LineChart`	Daily cash flow

2.8 Additional Dependencies

Package	Purpose
`@radix-ui/*`	Headless primitives: Dialog, Select, Popover, Dropdown, Checkbox

Package	Purpose
`tailwindcss 3`	CSS utilities consuming CSS custom property themes
`date-fns 3`	Date calculations (calendar, analytics, deadlines)
`i18next + react-i18next`	RU/EN localization, live language switching
`react-dnd 16`	Drag-and-drop: task and subtask reordering
`electron-builder 24`	Windows NSIS installer packaging
`concurrently`	Parallel Vite + Electron dev mode startup

3 Architecture

3.1 Monorepo Structure

The project is organized as an npm workspace with three packages:

```
toXo/
|-- package.json            (workspaces: main, preload, renderer)
|-- electron-builder.json5
|-- tsconfig.base.json
|-- main/
|   `-- src/
|       |-- index.ts         (app lifecycle, BrowserWindow)
|       |-- windowManager.ts (DWM titlebar, acrylic)
|       |-- tray.ts
|       |-- singleInstance.ts
|       |-- notificationScheduler.ts
|       |-- demoData.ts
|       |-- ipc/
|       |   |-- tasks.ipc.ts
|       |   |-- categories.ipc.ts
|       |   |-- analytics.ipc.ts
|       |   |-- finance.ipc.ts
|       |   `-- settings.ipc.ts
|       `-- db/
|           |-- database.ts
|           |-- migrations.ts
|           |-- taskRepo.ts
|           |-- categoryRepo.ts
|           |-- analyticsRepo.ts
|           `-- financeRepo.ts
|-- preload/
|   `-- src/preload.ts       (contextBridge -> window.electronAPI)
`-- renderer/
    `-- src/
        |-- main.tsx
```

```
        |-- app/              (App.tsx, ThemeProvider, i18nProvider)
        |-- pages/            (tasks/, analytics/, calendar/, finance
           /, settings/)
        |-- widgets/          (AppSidebar, TaskEditor, CommandPalette,
            ...)
        |-- features/         (CategoryBar, Topbar, filter-tasks, ...)
        |-- entities/         (task/store.ts, category/store.ts)
        `-- shared/
            |-- api/ipc.ts    (typed wrappers for window.electronAPI)
            |-- i18n/         (ru.ts, en.ts)
            |-- lib/          (undoRedo.ts, dateUtils.ts, sounds.ts)
            `-- ui/           (Button, Dialog, DateTimePicker,
               ColorPicker)
```

3.2 Data Flow

IPC Principle

The renderer **never** imports `electron` directly. All access to native functionality goes through `window.electronAPI`, which the preload script exposes via `contextBridge`. Every call in the renderer is wrapped in a typed function inside `shared/api/ipc.ts`.

Data call chain:

```
Renderer (React)
  -> ipc.tasks.list()          // shared/api/ipc.ts
  -> window.electronAPI.tasks.list()
  -> ipcRenderer.invoke('tasks:list')
  -> ipcMain.handle('tasks:list')   // main/ipc/tasks.ipc.ts
  -> db.tasks.listTasks()           // main/db/taskRepo.ts
  -> better-sqlite3 .prepare().all()
```

3.3 Main Process

3.3.1 index.ts — App Lifecycle

- `app.whenReady()` → database init → IPC handler registration → window and tray creation
- `app.requestSingleInstanceLock()` → second launch focuses the existing window
- `app.on('before-quit')` → graceful shutdown of the notification scheduler

3.3.2 windowManager.ts — Window and Visual Effect

Creates a `BrowserWindow` with `transparent: true`, `frame: false`, `show: false`. Launch sequence:

1. `ready-to-show`: call `applyAcrylic(win)` → `SetWindowCompositionAttribute` (sets acrylic on the hidden window)
2. `win.show()` → DWM reads the HWND attributes and renders acrylic from the first frame
3. On maximize and restore: `applyAcrylic` is called again

3.3.3 notificationScheduler.ts — Notifications

`setInterval(60_000)` polls the database:

- `taskRepo.getDueReminders()` → fires `new Notification(...).show()`, marks `remind_shown=1`
- `taskRepo.getOverdueDeadlines()` → deadline overdue notification
- Pushes IPC events `notification:reminderFired` / `notification:deadlineFired` to the renderer (for sound playback)

3.4 Preload

`preload/src/preload.ts` uses `contextBridge.exposeInMainWorld('electronAPI',` where `api` is an object of typed `ipcRenderer.invoke` wrappers for all channels. The full type surface is exported as `ElectronAPI` and reused in the renderer.

3.5 Renderer

Entry point: `renderer/src/main.tsx`. Root component: `App.tsx`. Startup initialization (`useEffect`):

1. `fetchTasks()` + `fetchCategories()` in parallel
2. Check `tour_suppressed` → deferred tutorial display
3. Minimum 4 s delay (splash screen display time)
4. Check `welcome_shown` → `ensureWelcomeTask()` → `selectTask(id)` on first launch
5. `setAppReady(true)` → `LaunchOverlay` begins fade-out

4 Database

4.1 File Paths

- **Database:** `%APPDATA%\toxo\tasks.sqlite`
- **Settings:** `%APPDATA%\toxo\settings.json`

Backward Compatibility

The database schema is fully identical to version 1.x (Python/PyQt6). An existing `tasks.sqlite` file opens in the new version without any modifications.

4.2 Table Schema

4.2.1 tasks

Column	Type	Description
`id`	INTEGER PK	Autoincrement
`title`	TEXT	Task title
`description`	TEXT	HTML content from TipTap editor
`is_done`	INTEGER	0/1
`priority`	INTEGER	0 none, 1 low, 2 medium, 3 high
`is_pinned`	INTEGER	0/1, pinned task
`category_id`	INTEGER FK	Reference to `categories.id`
`tags`	TEXT	Comma-separated tags
`recurrence`	TEXT	`daily` / `weekly` / `monthly` / NULL
`reminder_at`	TEXT	ISO 8601, reminder time
`deadline_at`	TEXT	ISO 8601, deadline
`remind_shown`	INTEGER	0/1, notification already shown
`is_archived`	INTEGER	0/1
`created_at`	TEXT	ISO 8601
`completed_at`	TEXT	ISO 8601 / NULL

4.2.2 subtasks

Column	Type	Description
`id`	INTEGER PK	Autoincrement
`task_id`	INTEGER FK	Reference to `tasks.id` (CASCADE DELETE)
`title`	TEXT	Subtask text
`is_done`	INTEGER	0/1

Column	Type	Description
`position`	INTEGER	Order in list

4.2.3 categories

Column	Type	Description
`id`	INTEGER PK	Autoincrement
`name`	TEXT	Category name
`color`	TEXT	HEX color, e.g. `#0a84ff`
`position`	INTEGER	Order in list

4.2.4 Finance Tables

Table	Contents
`fin_accounts`	Accounts: name, type (checking/savings/cash/credit/investment), balance, currency, color
`fin_categories`	Transaction categories: name, type (income/expense), color, icon
`fin_transactions`	Transactions: type (`income/expense/transfer`), amount, currency, category_id, account_id, **to_account_id** (transfers), date, title, note, tags, is_recurring
`fin_budgets`	Budgets: category_id, limit_amount, period (monthly/weekly)
`fin_goals`	Savings goals: name, target_amount, current_amount, currency, color, deadline

4.3 AppDatabase Class

`main/src/db/database.ts` composes four repositories:

```
export class AppDatabase {
  readonly db:         Database.Database;
  readonly tasks:      TaskRepo;
  readonly categories: CategoryRepo;
  readonly analytics:  AnalyticsRepo;
  readonly finance:    FinanceRepo;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    runMigrations(this.db);      // migrations.ts v0..v13
    this.db.pragma('journal_mode = WAL');
```

```
    this.db.pragma('foreign_keys = ON');
    this.tasks      = new TaskRepo(this.db);
    this.categories = new CategoryRepo(this.db);
    this.analytics  = new AnalyticsRepo(this.db);
    this.finance    = new FinanceRepo(this.db);
  }
}
```

5 Application Pages

5.1 Tasks

The main page. Split into three columns:

1. **CategoryBar** — horizontal strip of colored categories + add button
2. **TaskList** — task list with filtering, sorting, search, drag-and-drop
3. **TaskEditor** — right panel: title, TipTap, priority, pin, deadline, reminder, recurrence, tags, subtasks, action buttons

The Topbar above the list contains filter buttons (All / Active / Done / Today / Archive), a search field, and a sort dropdown.

5.1.1 Multi-select

- **Ctrl+click** — toggle individual task in/out of selection
- **Shift+click** — select range from the anchor (last plain-clicked task) to the current
- **Ctrl+Shift+click** — add range to existing selection
- **Ctrl+A** — select all visible tasks (uses `e.code = 'KeyA'`, layout-independent)

State lives in `useTaskStore`: `selectedTaskIds: number[]`, `lastSelectedIdRef` (anchor) is a `useRef` in the component.

5.1.2 Inline Rename (F2)

Pressing `F2` or double-clicking a task title in `TaskItem` enters rename mode: the title span is replaced by an `<input>` with an accent-colored underline. `Enter` — save, `Escape` — cancel, `blur` — auto-save. Calls `useTaskStore.updateFields`.

5.1.3 Task Context Menu

Right-click or the "···" button on a card opens a menu:

- Open / Duplicate / Delete
- Pin / Unpin
- Set Priority 1 / 2 / 3
- Archive / Unarchive

5.1.4 Undo/Redo

`useHistoryStore` maintains a stack of `{undo, redo}` pairs. Each destructive operation (deletion, archiving) pushes a task snapshot and a `tasks:restore` call for recovery. Ctrl+Z and Ctrl+Y execute the corresponding functions.

5.2 Analytics

Six Recharts graphs and four KPI cards. Period is selected via buttons: **7 days / 30 days / 90 days / Custom**. When "Custom" is selected, from/to date inputs appear and data is re-fetched on confirmation.

Chart	Description
Completed per Day	AreaChart: tasks completed per day
Created vs Completed	BarChart: created vs completed over the period
Status Distribution	PieChart: active / done / archived
By Weekday	BarChart: average productivity by day of week
By Category	BarChart: tasks completed per category
Priority Distribution	RadialBarChart: priority breakdown

KPI cards: **Streak** (consecutive days with completed tasks), **Done Total**, **Completion Rate**, **Best Weekday**.

5.3 Calendar

Monthly grid and a day detail panel:

- Each cell shows up to 2 task chips for tasks with a deadline on that day
- Clicking a cell → loads that day's tasks in the right panel
- **Quick create** — input field at the bottom of the panel; `Enter` or the "+" button
- **Quick edit** — clicking a chip in the grid *or* a row in the panel activates edit mode with an input field, Save/Cancel buttons (`Enter` / `Escape`)
- **Drag-and-drop** — drag a task from `TaskList` onto a calendar cell to set its deadline to 12:00 on that day. Since pages are on separate tabs, hovering a dragged task over the **Calendar** sidebar button for 600 ms triggers automatic page switch (drag-to-navigate)
- **Open in Tasks button (↗)** — each task row in the day panel has this button; clicking it switches to the Tasks tab and selects the task in the editor
- "Today" button returns to the current date

5.4 Finance

Five tabs:

1. **Overview** — total account balance, cash flow charts and expense-by-category
2. **Transactions** — table with period and type filter, add/delete
3. **Accounts** — account cards with balance; add via dialog
4. **Budgets** — progress bar of spending against the limit for the current period
5. **Goals** — savings goal cards with progress; "Add funds" button

Account Transfers. A `transfer` transaction debits one account and credits another. The Quick-add panel gains a third "Transfer" button; selecting it reveals an additional "Target account" selector. Implemented atomically via an SQLite transaction (`FinanceRepo.addTransfer`). DB schema extended in migration v13: `ALTER TABLE fin_transactions ADD COLUMN to_account_id INTEGER`.

When `clearAll` or `loadDemo` is called from settings, the page receives a custom event `finance:reload` and reloads all data.

5.5 Settings

Section	Options
Appearance	Theme (Dark / Light / Glass), language (RU / EN)
Notifications	Enable/disable, sound selection (6 presets + test)
Data	Load demo data, Clear all, Export JSON/CSV, Import JSON
About	Version, repository link, Start Tour button

6 IPC API Reference

All channels are registered via `ipcMain.handle`. The renderer accesses them through `window.electronAPI` (preload).

6.1 tasks

Channel	Description
`tasks:list`	List all tasks
`tasks:get`	Get task by id
`tasks:create`	Create a task `{title?, description?, categoryId?}`
`tasks:update`	Update title + description
`tasks:updateFields`	Update arbitrary fields
`tasks:delete`	Delete a task
`tasks:setDone`	Set is_done
`tasks:setPriority`	Set priority (0–3)
`tasks:setPinned`	Pin/unpin
`tasks:setRecurrence`	Set recurrence
`tasks:setReminder`	Set reminder (ISO)
`tasks:setDeadline`	Set deadline (ISO)
`tasks:setCategory`	Assign to category
`tasks:setTags`	Set tags (string)
`tasks:reorder`	Reorder tasks (array of ids)
`tasks:archive`	Archive/unarchive
`tasks:archiveCompleted`	Archive all done tasks older than a date
`tasks:clearArchive`	Delete all archived tasks
`tasks:duplicate`	Duplicate a task
`tasks:restore`	Restore from snapshot (undo)
`tasks:getByDeadlineDate`	Tasks with deadline on date (YYYY-MM-DD)
`tasks:logEvent`	Log event to analytics
`tasks:exportJson`	Export to JSON file
`tasks:exportCsv`	Export to CSV file
`tasks:importJson`	Import from JSON file
`tasks:loadDemo`	Load demo data (language-aware)
`tasks:clearAll`	Delete all data (tasks + finance)
`tasks:ensureWelcomeTask`	Find or create the welcome task

6.2 subtasks

Channel	Description
`subtasks:list`	Subtasks by taskId
`subtasks:add`	Add a subtask
`subtasks:setDone`	Set is_done
`subtasks:rename`	Rename subtask
`subtasks:delete`	Delete subtask
`subtasks:reorder`	Reorder subtasks
`subtasks:counts`	All/done counters for all tasks

6.3 Other Channel Groups

Group	Channels
`categories:*`	list, add, update, delete, reorder
`analytics:*`	getKpi, completedPerDay, created-VsCompleted, statusDistribution, completedByWeekday, completed-ByCategory, priorityDistribution
`fin:accounts:*`	list, add, update, delete
`fin:categories:*`	list, add, update, delete
`fin:tx:*`	list, add, **transfer**, update, delete, summary, dailyFlow, expenseByCategory, cumulativeBalance, totalBalance
`fin:budgets:*`	list, add, update, delete, getSpent
`fin:goals:*`	list, add, update, delete, addTo
`settings:*`	get, set, getAll
`shell:*`	showSaveDialog, showOpenDialog

6.4 Push Events (main → renderer)

Channel	Description
`notification:reminderFired`	Reminder triggered → play sound
`notification:deadlineFired`	Deadline triggered → play sound
`window:maximized`	Window maximized/restored (boolean)
`tray:newTask`	“New Task” pressed in tray menu

7 Keyboard Shortcuts

Shortcut	Action
`Ctrl+N`	New task
`Ctrl+K`	Open / close command palette
`Ctrl+Z`	Undo last action
`Ctrl+Shift+Z` / `Ctrl+Y`	Redo undone action
`Ctrl+D`	Duplicate selected task
`Ctrl+P`	Pin / unpin task
`Ctrl+1`	Set priority Low
`Ctrl+2`	Set priority Medium
`Ctrl+3`	Set priority High
`Ctrl+A`	Select all visible tasks (multi-select)
`Ctrl+click`	Toggle individual task in/out of selection
`Shift+click`	Select range from anchor to current task

Shortcut	Action
`F2`	Rename focused task inline
`Delete`	Delete selected task
`Escape`	Deselect / close dialog / cancel rename
`Enter`	Confirm input (quick create / rename / edit)

8 Build and Distribution

8.1 Scripts

Command	Description
`npm run dev:app`	Dev mode: Vite dev server + Electron with hot reload
`npm run build`	Full production build: renderer → preload → main
`npm run package`	`build` + `electron-builder` → NSIS .exe installer
`npm run rebuild-native`	Rebuild `better-sqlite3` for the current Electron version

8.2 electron-builder

`electron-builder.json5` configuration:

- Target: **NSIS** (Windows x64)
- `asarUnpack`: `node_modules/better-sqlite3/**/*` — the native `.node` file must reside outside the asar archive
- Icon: `build/app_icon.ico`
- AppId: `com.theoverforge.toxo`

8.3 Fresh Machine Setup

```
git clone https://github.com/TheOverforge/toxo
cd toxo
npm install
npm run rebuild-native     # rebuild better-sqlite3
npm run dev:app            # development mode
# or
npm run package            # create installer
```

System Requirements

- Windows 10 version 1903+ or Windows 11
- Node.js ≥ 18 LTS
- Python 3 (only for building native modules via `node-gyp`)
- Visual C++ Build Tools

9 Settings File (settings.json)

`%APPDATA%\toxo\settings.json` stores user preferences as JSON:

Key	Type	Description
`theme`	string	`"dark"` / `"light"` / `"glass"`
`language`	string	`"ru"` / `"en"`
`notification_sound`	string	Reminder sound key
`notifications_enabled`	boolean	Notifications enabled
`welcome_shown`	boolean	Welcome task already shown
`tour_suppressed`	boolean	Tutorial disabled
`color_recent`	string[]	Recently used colors
`color_favorites`	string[]	Favorite colors
`custom_fonts`	string[]	User-added font names